

COMPANY	WEBTEZZA (PVT) LTD
POSITION	Software Engineering Intern (Full-Stack / React / Next.js)
ASSESSMENT TYPE	Practical Engineering Challenge — ApparelFlow ERP Execution System
FOCUS MODULE	Production Batch Verification & Sewing Queue Gate (Full-Stack RBAC)
DURATION & WORKLOAD	4 Calendar Days from issue date (Target: 28–32 focused engineering hours)
TECH STACK	Next.js or React + Node.js/Express, Relational DB (PostgreSQL / Supabase / SQLite), Cloud Deployed

1. INTRODUCTION

Thank you for applying for the Software Engineering Intern position at Webtezza (Pvt) Ltd. As the primary practical step in our technical selection process, you are invited to complete this production-grade engineering challenge.

In the modern AI-assisted software era, exceptional software engineers are not evaluated by their ability to generate syntax from prompts. We evaluate your engineering depth: how you comprehend mission-critical business requirements, model persistent relational schemas, enforce non-bypassable server-side security boundaries, write automated test suites, audit AI-generated code, and deploy resilient web software to the cloud.

2. SYSTEM CONTEXT: APPARELFLOW ERP

ApparelFlow is Webtezza’s enterprise manufacturing resource planning platform engineered specifically for commercial garment production facilities. In industrial apparel manufacturing, every garment design (e.g., Casual Blouse, Office Shirt, Crop Top) is structured around a strict **Production Recipe (Bill of Materials)** containing specific cut parts (e.g., Front Body, Back Body, Left Sleeve, Right Sleeve, Collar, Cuffs, Pockets).

Bulk fabric rolls are measured, spread, cut, and bundled inside the **Cutting Department** before being handed over to the assembly floor where over 100 sewing machine operators assemble garments. Plant profitability and operational throughput depend entirely on cutting accuracy.

3. THE REAL-WORLD ENGINEERING PROBLEM

In garment manufacturing, the Cutting Department is the single most critical quality checkpoint. If cut bundles contain fabric defects, component shortages, or count mismatches, and those bundles reach the high-speed sewing floor, the entire assembly line grinds to a halt. Incomplete garments pile up, expensive fabric rolls are wasted, and international shipping dates are missed.

To prevent this operational failure, the ERP mandates an unyielding, non-bypassable checkpoint: **The Cutting Operations & Gatekeeper Verification Terminal**. Under factory standard operating procedures (SOP), **zero unverified, mismatched, or shortage batches may ever enter the Sewing Queue**. The application must enforce this as a server-side hard stop.

CRITICAL SYSTEM BOUNDARY (SERVER-ENFORCED HARD STOP)
A cutting batch can **NEVER** proceed to sewing assembly without explicit component-by-component verification and digital sign-off by an authorized Cutting Verifier. If even a single component has a shortage, backend validation must reject the approval and physically prevent the batch from entering the Sewing Queue.

4. YOUR PRACTICAL CHALLENGE & SCOPE BOUNDARY

You are tasked with engineering a fully functional, production-deployed full-stack web application that implements the **Cutting Operations & Gatekeeper Verification Terminal** for ApparelFlow ERP.

Scoping Boundary: You are intentionally **not** expected to build the entire 23-module ERP suite. Instead, you must deliver a robust, full-stack implementation of this mission-critical checkpoint—demonstrating real-time quantity validation, Role-Based Access Control (RBAC), a persistent relational database, server-side tamper protection, and passing automated test suites.

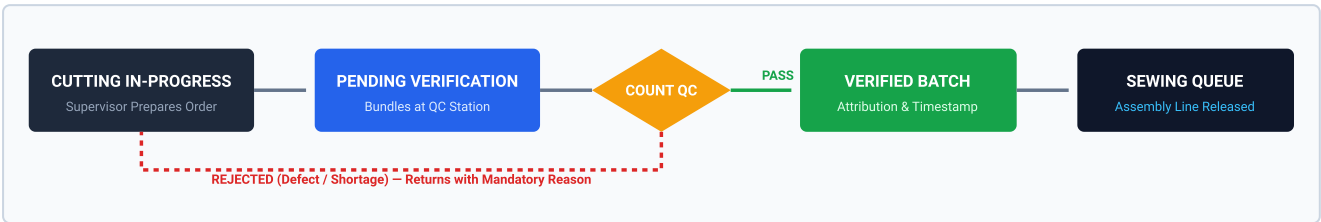
5. OPERATIONAL PERSONAS & ROLE-BASED ACCESS CONTROL (RBAC)

Your application must support real authentication and role enforcement across three distinct factory roles. Provide a visible **Role Switcher / Demo Credential Panel** allowing the evaluator to test the system under each persona:

1. Cutting Supervisor Role: cutting_supervisor Creates cutting orders from recipes, sets batch quantities, logs fabric yards, and tracks cutting progress. Restriction: Cannot verify batches (separation of duties). Cannot access the Sewing Queue.	2. Cutting Verifier Role: cutting_verifier Isolated QC checkpoint. Counts physical parts per recipe, triggers traffic lights, and approves or rejects batches. Restriction: Cannot create cutting orders or edit recipes. Cannot access the Sewing Queue.	3. Sewing Supervisor Role: sewing_supervisor Receives verified batches on the assembly floor, reviews verifier audit notes, and initiates sewing operations. Restriction: Strictly blocked from seeing unverified, pending, or rejected cutting orders.
---	---	---

6. MANUFACTURING STATE MACHINE ARCHITECTURE

Your state management and backend database layer must implement the following deterministic state pipeline:



IMMUTABLE SERVER-SIDE AUDIT TRAIL REQUIREMENT

When an order transitions to VERIFIED, the backend must permanently write the verifier’s user ID, timestamp, component count variances, and fabric wastage percentage to the database. These values are immutable and accompany the batch into the sewing queue.

7. CORE FUNCTIONAL SPECIFICATIONS & COMPONENT ENGINE

7.1 Pre-Seeded Production Recipes (Bill of Materials)

To allow instant end-to-end evaluation, seed your database with at least two real garment production recipes:

Recipe A: "Casual Blouse" (Code: REC-BL01)

Category: Blouse | **Std Fabric:** 1.8 yds/piece | **Wastage Cap:** 5.0%

- Front Body Panel: 1 pcs / garment
- Back Body Panel: 1 pcs / garment
- Sleeves (Left & Right): 2 pcs / garment
- Collar & Stand: 1 pcs / garment
- Sleeve Cuffs: 2 pcs / garment

Recipe B: "Crop Top" (Code: REC-CT02)

Category: Crop Top | **Std Fabric:** 1.1 yds/piece | **Wastage Cap:** 8.0%

- Front Chest Panel: 1 pcs / garment
- Back Support Panel: 1 pcs / garment
- Neck Binding Strip: 1 pcs / garment
- Hem Elastic Casing: 1 pcs / garment
- Side Strap Accents: 2 pcs / garment

7.2 Cutting Order Management (Cutting Supervisor)

The Cutting Supervisor initiates production batches through an order creation interface:

- **Required Inputs:** Recipe ID, Target Batch Quantity (e.g., 50 units), Fabric Roll ID (e.g., FAB-ROLL-882), and Actual Fabric Used (yards).
- **Multiplier Engine:** System dynamically derives Expected Component Counts (e.g., 50 garments × 2 cuffs = 100 cut cuffs expected).
- **Order Submission:** Submitting transitions the order to PENDING_VERIFICATION.

7.3 Verification Terminal & Traffic-Light Status Matrix (Cutting Verifier)

The Verifier counts physical pieces for every component. The system evaluates status in real time:

Status Flag	Condition Formula	System Behaviour & Action Rule
GREEN (MATCH)	Actual == Expected	Exact component match. Satisfies verification requirement.
YELLOW (EXCESS)	Actual > Expected	Surplus pieces recorded. Flagged for return or safety margin; batch may proceed.
RED (SHORTAGE)	Actual < Expected	DEFECT SHORTAGE. Garment assembly incomplete. Approve action blocked.

7.4 The Hard-Stop Gatekeeper Rule

STRICT PASS-FAIL VALIDATION ENFORCEMENT

If **ANY** single component is flagged **RED**, the **"Approve Batch"** action must be strictly disabled on the UI and rejected on the API. The Verifier may only click **"Reject Batch"** with a mandatory reason note, returning the batch to the Supervisor for re-cutting.

7.5 Sewing Queue Handoff & Wastage Analytics

Once verified, the batch status becomes VERIFIED and is released to the Sewing Queue:

- **Sewing Floor View:** Only verified orders appear. Sewing Supervisor can inspect piece counts, verifier attribution, and click "Start Sewing Assembly".
- **Wastage Calculation:** The backend computes and stores fabric variance:
Fabric Wastage % = [(Actual Fabric Used - Expected Fabric) ÷ Expected Fabric] × 100

8. MINIMUM RELATIONAL DATABASE SCHEMA

Your application must use a real persistent database (PostgreSQL, MySQL, Supabase, Neon, or SQLite). You may refine table structures, but the schema must represent at least the following relational entities:

Table Entity	Key Attributes & Data Types	Relationships
users	id, email, password_hash, role, full_name, created_at	Has many orders & verifications
recipes	id, recipe_code, name, category, std_fabric_yards, wastage_cap	Has many components & orders
recipe_components	id, recipe_id, component_name, pieces_per_garment, image_url	Belongs to recipe
cutting_orders	id, order_no, recipe_id, target_qty, fabric_roll_id, actual_fabric_yds, status, created_by, timestamps	Belongs to recipe & user; has many items
verification_items	id, order_id, component_id, expected_qty, actual_qty, status (GREEN/YELLOW/RED)	Belongs to order & component
verification_logs	id, order_id, verifier_id, decision (APPROVED/REJECTED), rejection_note, wastage_pct, timestamp	Belongs to order & verifier

9. SERVER-SIDE TAMPER PROTECTION & SECURITY RULES

Security cannot exist purely on the client. Disabled buttons, hidden tabs, and frontend redirects are not security boundaries. Your backend API / Server Actions must enforce:

- **Server-Side RBAC:** If a user with role `cutting_supervisor` sends a direct POST request to approve a verification, the server must reject it with `403 Forbidden`.
- **Server-Side Hard Stop:** If an approval request is received for an order where any component is `RED`, missing, or uncounted, the backend must reject the request with `422 Unprocessable Entity`.
- **Query Isolation:** The endpoint serving the Sewing Queue (e.g., `GET /api/sewing/queue`) must enforce `WHERE status = 'VERIFIED'` at the database level. Manipulating URL params must not leak unapproved orders.
- **Authenticated Context:** Verifier identity and audit timestamps must be derived from the server session/JWT—never trusted from client request bodies.

SECURITY CONTRACT: NO CLIENT-SIDE CHEATING

Evaluators will inspect your network tab and test endpoints directly via Postman/cURL. Bypassing frontend UI validation to submit illegal status updates must fail cleanly on the server.

10. AUTOMATED TESTING SPECIFICATIONS

Provide automated tests (using Jest, Vitest, Playwright, or your framework's native test runner) verifying the core domain rules. Your test suite must cover at least:

- **Test 1:** An order with all `GREEN` components can be approved by an authenticated Verifier.
- **Test 2:** An order containing at least one `RED` (shortage) component blocks approval and returns an error.
- **Test 3:** Rejecting an order without a reason note is rejected by backend validation.
- **Test 4:** Non-verifier roles receive `403 Forbidden` when attempting verification approval.
- **Test 5:** Unapproved orders never appear in the Sewing Queue database query.

11. UI USABILITY, CONTRAST & NON-FUNCTIONAL STANDARDS

ZERO-TOLERANCE DEFECT: UI INPUT CONTRAST & ACCESSIBILITY

In our recent system UAT audit, severe contrast defects occurred where form input text rendered white-on-white, making text fields, search bars, and dropdown menus completely invisible. **Your application must exhibit high-contrast, fully legible text across all input states, focus states, and dropdown elements.**

State Persistence Across Reloads

All created orders, counts, status transitions, and audit logs must persist across page reloads in your persistent cloud database. Data must never vanish on refresh.

Defensive Input Guards

Inputs must strictly reject negative numbers, decimals, non-numeric strings, and empty payloads. Forms must supply immediate inline visual errors.

12. AI USAGE & ENGINEERING JUDGMENT PROTOCOL

In this assessment, the use of modern AI engineering tools (ChatGPT, Claude, Cursor, GitHub Copilot, v0, etc.) is **fully permitted and encouraged**.

However, blindly accepting raw AI output without human architectural scrutiny is unacceptable for production engineering. AI models frequently introduce subtle flaws: client-side bypassable RBAC logic, component re-render loops, brittle numeric coercions, and poor accessibility/contrast defaults.

You must submit an `AI_OPTIMIZATION_REPORT.md` in your repository addressing the four mandatory sections below:

Report Section	What You Must Candidly Document
1. Tools & Prompting	Which AI tools were used, and for which tasks (scaffolding, schema design, styling, test generation)?
2. Flawed / Broken AI Code	Identify at least 2 specific instances where AI generated buggy, insecure, or sub-optimal code (e.g., bypassable validation, contrast issues, state sync bugs).
3. Human Refactoring	How did you rewrite, optimize, and harden the code using your own engineering judgment?
4. Defensive Architecture	How did you structure the state machine and API guards to prevent unauthorized status overrides?

13. 4-DAY MILESTONE & EXECUTION SCHEDULE

Timeline	Target Milestone	Expected Deliverables
Day 1	Architecture, Database & Repo	Setup project repo, configure relational DB, seed recipes, deploy skeleton to cloud.
Day 2	Supervisor & Order Engine	Build Role Switcher, order creation modal, dynamic component multiplier, and validation.
Day 3	Verifier Terminal & Server Hard Stop	Build Verifier workspace, traffic-light logic, server-side gatekeeper, and rejection note flow.
Day 4	Sewing Queue, Tests & AI Report	Build Sewing Queue, write automated tests, check contrast, and author AI optimization log.

14. MANDATORY SUBMISSION REQUIREMENTS

Deliverable Item	Mandatory	Submission Format / Location
Live Deployed URL	STRICT YES	Public cloud URL (Vercel, Netlify, Render, Railway). Must be live and testable.
GitHub Repository	STRICT YES	Public repository with atomic commit history demonstrating iterative development.
AI Optimization Report	STRICT YES	AI_OPTIMIZATION_REPORT.md inside repository root.
Project README.md	REQUIRED	Architecture summary, schema documentation, and demo credentials for all 3 roles.
Automated Test Suite	REQUIRED	Passing unit/integration test suite executable via <code>npm test</code> or equivalent.

15. COMPREHENSIVE EVALUATION CRITERIA (100% WEIGHTED RUBRIC)

Evaluation Dimension	Weight	Specific Criteria Evaluated
Domain & Business Logic	15%	Accurate modeling of apparel recipes, component multipliers, and production flow.
Gatekeeper Hard Stop	20%	Server-enforced traffic-light logic; strict blocking of shortage batches from sewing.
Role Isolation & RBAC	15%	Clean separation of duties; endpoints enforce permissions server-side.
Database & Architecture	15%	Relational database modeling, predictable state management, and data persistence.
UI Contrast & Usability	15%	Zero low-contrast defects; clear input styling, legible dropdowns, and responsive layout.
Automated Tests & Edge Cases	10%	Passing test suite for core rules; defensive handling of invalid inputs and rejections.
AI Candor & Optimization	10%	Honest, technical documentation of AI assistance, hallucinations caught, and refactors.

16. EVALUATOR 5-MINUTE TECHNICAL AUDIT CHECKLIST

Webtezza technical evaluators will perform the following 5-minute verification on your submitted URL:

- **Contrast Audit:** Click every input and dropdown. Verify dark, legible text against light backgrounds.
- **RBAC Check:** Switch to Verifier; verify order creation button is hidden. Switch to Sewing; verify unverified batches are invisible.
- **Shortage Hard Stop:** In Verifier Terminal, enter a shortage count. Confirm the "Approve Batch" button is disabled and backend blocks it.
- **Sewing Handoff:** Approve a valid GREEN batch. Verify it appears in Sewing Queue. Refresh browser; confirm persistence.
- **Code & AI Report:** Inspect GitHub commits, check test passes, and evaluate whether the candidate genuinely audited AI code.

WHAT WE REALLY WANT TO SEE

We are not testing who can run the fastest prompt. We are looking for engineers who take ownership of the software they deliver —engineers who understand physical operational constraints, build reliable safeguards, and ship resilient software. Good luck!